

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ Zinck
Nom d'usage ▶ Zinck
Prénom ▶ Nicolas
Adresse ▶ 2 rue Louis Pasteur Mehun-Sur-Yèvre 18500

Titre professionnel visé

Développeur(se) web et web mobile

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile	p.	5
▶ Installation et configuration de l'environnement de travail	p.	5
▶ Réaliser des interfaces utilisateur statique web ou web mobile	p.	10
▶ Développer la partie dynamique des interfaces utilisateur web ou web mobile	p.	14
Développer la partie Back-end d'une application web ou web mobile	p.	18
▶ Développer des composants d'accès aux données SQL et NoSQL	p.	18
▶ Développer des composants métier côté serveur	p.	21
Titres, diplômes, CQP, attestations de formation <i>(facultatif)</i>	p.	23
Déclaration sur l'honneur	p.	24
Documents illustrant la pratique professionnelle <i>(facultatif)</i>	p.	25
Annexes <i>(Si le RC le prévoit)</i>	p.	26

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer la partie Front-end d'une application web ou web mobile

Exemple n°1 ► Installation et configuration de l'environnement de travail

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Installation de l'environnement de travail

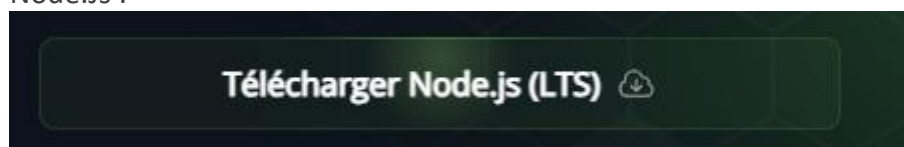
Node.Js :

Avant de commencer le développement du projet LocaFestia, nous avons installé l'environnement de travail nécessaire au bon fonctionnement du framework Next.Js et du préprocesseur CSS Tailwind CSS.

La première étape a été l'installation de Node.Js, une plateforme permettant d'exécuter du JavaScript côté serveur et nécessaire au bon fonctionnement des outils modernes de développement web. Pour cela, nous avons téléchargé la dernière version LTS de Node.Js depuis le site officiel <https://nodejs.org/fr>, ce qui a également installé npm (Node Package Manager), Indispensable pour gérer les dépendances du projet

Installation de Node.Js :

Sur la page d'accueil du site <https://nodejs.org/fr>, il est proposé de télécharger la version LTS de Node.Js :



Une fois ce bouton cliqué, le téléchargement du fichier « node-v22.16.0-x64.msi » se lance. Une fois télécharger terminer, il suffit de double cliquer sur ce fichier pour lancer l'installation. Ensuite une fenêtre nous informe que nous pouvons poursuivre l'installation de Node.Js en cliquant sur le bouton « Next »

Node.js Setup

Welcome to the Node.js Setup Wizard



The Setup Wizard allows you to change the way Node.js features are installed on your computer or to remove it from your computer. Click Next to continue or Cancel to exit the Setup Wizard.

Back Next Cancel

Pour la suite de l'installation la prochaine fenêtre présente l'étape de personnalisation de l'installation de Node.Js sur Windows, où l'utilisateur peut sélectionner les composants comme le runtime Node.Js et le gestionnaire de paquets NPM. L'option « Add to PATH » est cruciale pour l'utilisation en ligne de commande. Elle montre le choix des fonctionnalités avec de passer à l'étape suivante de l'installation

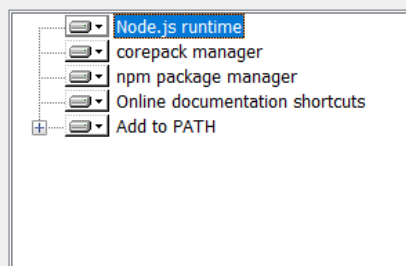
Node.js Setup

Custom Setup

Select the way you want features to be installed.



Click the icons in the tree below to change the way features will be installed.



Install the core Node.js runtime (node.exe).

This feature requires 0KB on your hard drive.

Reset

Disk Usage

Back

Next

Cancel

La prochaine étape de l'installation de Node.Js propose d'installer les outils nécessaires (Python et visual Studio Build Tools) pour compiler les modules Node.Js natif écrits en C/C++. L'utilisateur a la choix entre un installation automatique ou une installation manuelle en suivant les instruction de node-gyp

Une fois l'installation terminé nous devons initialisée un projet en Next.Js avec la commande suivant :

```
npx create-next-app@latest locafestia
```

Cela a généré la structure de base d'un projet Next.Js, avec tous les fichiers et configurations

nécessaires. Nous avons ensuite installée tailWind CSS avec les commandes suivantes :

```
>npm install -D tailwindcss|  
  
>npx tailwindcss init -p|
```

Flutter :

Avant de démarrer le développement de l'application mobile SportGuess, nous avons mis en place un environnement de travail adapté au Framework Flutter, qui permet de créer des applications multiplateformes (Android, IOS, Web, Desktop) avec un seul code source.

Installation de Flutter et configuration initiale

J'ai tout d'abord créé un dossier spécifique destiné à regrouper mes projets de développement, situé à ce chemin-là :

« c:\Users\nicolas\developpement »

Depuis le site officiel Flutter.dev, j'ai téléchargée l'archive contenant le SDK Flutter, puis je l'ai extrait dans ce dossier. Cela a permis d'avoir une structure claire et organisée dès le départ.

Voici le lien pour télécharger le SDK Flutter pour Windows.

« <https://docs.flutter.dev/get-started/install/windows/web> »

Configuration Système

Après l'extraction, j'ai ajouté le chemin suivant à mes variables d'environnement (PATH) afin de pouvoir utiliser les commandes Flutter depuis n'importe quel terminal :

« c:\Users\nicolas\developpement \flutter\bin»

Une fois cela fait, j'ai utilisé la commande suivante pour vérifier que tout était correctement installé :

« flutter doctor »

Voici le résultat de la commande quand tout est bien installer :

```
C:\Users\znico\developpement\flutter\bin>flutter doctor  
  
A new version of Flutter is available!  
To update to the latest version, run "flutter upgrade".  
  
Doctor summary (to see all details, run flutter doctor -v):  
[✓] Flutter (Channel stable, 3.29.3, on Microsoft Windows [version 10.0.26100.4061], locale fr-FR)  
[✓] Windows Version (11 Famille 64-bit, 24H2, 2009)  
[✓] Android toolchain - develop for Android devices (Android SDK version 35.0.1)  
[✓] Chrome - develop for the web  
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.14.0)  
[✓] Android Studio (version 2024.3.2)  
[✓] VS Code (version 1.100.2)  
[✓] Connected device (3 available)  
[✓] Network resources  
  
• No issues found!
```

Cette commande permet de détecter les éventuelles erreurs de configuration. Elle m'a indiqué les étapes manquantes, comme l'installation d'un émulateur Android ou les composants nécessaires à la bonne exécution de Flutter.

Utilisation de l'émulateur Android avec Android Studio

Pour travailler et tester l'application SportGuess dans un environnement proche d'un appareil réel, j'ai choisi d'utiliser Android Studio comme émulateur. Après avoir installé Android Studio, j'ai configuré un Virtual Device via l'AVD Manager (Android Virtual Device), en choisissant un modèle de smartphone courant (Pixel 5 sous Android 13). Cela m'a permis de simuler le comportement de l'application sur un téléphone Android sans avoir besoin d'un appareil physique. L'émulateur s'intègre parfaitement avec Flutter et Visual Studio Code, permettant ainsi de lancer l'application directement depuis le terminal avec la commande « flutter run » et d'avoir un retour visuel immédiat sur mes développements et modification de code.

Mise en place de l'IDE

Pour le développement, j'ai choisi Visual Studio Code, léger et parfaitement compatible avec Flutter. J'y ai installé les extensions Flutter et Dart.

Intégration de Firebase

Lorsque j'ai intégré l'équipe sur le projet SportGuess, l'environnement Firebase était déjà en place. Le projet utilisait Firebase pour gérer l'authentification, la base de données Firestore, le système de notification push (via Firebase Messaging) ainsi que la gestion des compétitions. Mon rôle a consisté à prendre en main l'environnement existant, à m'assurer que les accès aux différentes ressources étaient sécurisés, et à tester et corriger les erreurs liées à l'utilisation de Firebase.

J'ai notamment travaillé sur la gestion des erreurs utilisateur (formulaires mal remplis, absence de connexion, permission), et j'ai participé à l'amélioration de la structure du code afin de mieux organiser les appels à Firebase dans des fonctions réutilisables. Cette prise en main m'a également permis de mieux comprendre les interactions entre l'application Flutter et les services back-end proposés par Firebase.

Conclusion

L'installation de l'environnement de travail a constitué une étape essentielle pour pouvoir contribuer efficacement au projet SportGuess. Malgré mon arrivée en cours de route, j'ai su rapidement configurer l'ensemble des outils nécessaires, notamment Flutter SDK et Android Studio, pour lancer l'application en local et réaliser les premiers tests. Cette mise en place m'a permis de me familiariser avec l'architecture du projet, de comprendre son fonctionnement global, et de commencer à développer de nouvelles fonctionnalités dans un environnement professionnel déjà structuré.

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

- Visual Studio Code
- Terminal intégré a mon éditeur de code
- Node.JS
- Tailwind css
- Next.Js
- SDK Flutter
- Android Studio

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul pour cette partie

4. Contexte

Nom de l'entreprise, organisme ou association ☐ *Dev Project & Co*

Chantier, atelier, service ► Installation de son environnement de travail

Période d'exercice ► Du : *09/04/2025* au : *30/06/2025*

5. Informations complémentaires (facultatif)

Activité-type 1 Développer la partie Front-end d'une application web ou web mobile

Exemple n°2 ► Réaliser des interfaces utilisateur statique web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Réaliser des interface utilisateur statique web ou web mobile :

Dans le cadre du projet LocaFestia, l'intégration de l'interface utilisateur a débuté par la création d'une structure statique fidèle à la maquette validée par le client. L'objectif principal était de poser les fondations visuelles de l'application à l'aide de composants React, en veillant à la cohérence graphique, à la responsivité et à la réutilisabilité du code.

J'ai utilisé Next.JS pour construire l'arborescence du projet, où chaque page du site correspond à un fichier « page.tsx » dans un dossier dédié (par exemple : /auth/login, /auth/register). Les éléments réutilisables comme le header et le footer ont été extraits en composants distincts placé dans un des dossiers spécifiques (components/nav), respectant ainsi l'architecture modulaire propre aux framework basé sur REACT.

Pour construire l'en-tête du site, j'ai réalisé une interface statique en utilisant des composants React et le système de routing de Next.Js. Le menu de navigation, par exemple, repose sur un tableau statique « NavItems » contenant les liens vers les différentes pages du site (Accueil, Salles, Gîtes, Contact). Chaque lien est généré dynamiquement à l'aide de la méthode « map() », ce qui permet une structure modulaire et facilement maintenable. L'ensemble de la mise en page est stylisé avec Tailwind CSS, sans recourir à des fichiers CSS externes, ce qui facilite la lecture et la modification du code. Des classes utilitaires telles que (Flex, Justify-between ou hover :bg-white) assurent une disposition responsive et une interaction fluide pour l'utilisateur. Le logo est affiché via le composant optimisé « Image » de Next.Js, garantissant de bonnes performances de chargement.

Voici un extrait de mon header :

DOSSIER PROFESSIONNEL (DP)

```
const Header = () => {
  const [user, setUser] = useState<User | null>(null);
  const [isMenuOpen, setIsMenuOpen] = useState(false);

  const navItems = [
    { name: "Accueil", href: "/" },
    { name: "Salles", href: "/salles" },
    { name: "Gîtes", href: "/gîtes" },
    { name: "Contact", href: "/contact" },
  ];

  useEffect(() => {
    const unsubscribe = auth.onAuthStateChanged((user) => {
      setUser(user);
    });

    return () => unsubscribe();
  }, []);

  return (
    <header className="flex flex-wrap justify-between w-full py-5 items-center text-white bg-layout sticky top-0 z-50">
      <div className="flex items-center">
        <div className="flex items-center w-auto id="logo">
          <Link href="/">
            <Image
              src="/logo/locafestiabg.png"
              alt="Logo de Loca Festia"
              width={80}
              height={80}
              className="align-baseline mx-5"
            />
          </Link>
        </div>

        <h1 className="text-4xl text-background font-ballet max-[768px]:mx-auto">
          Loca Festia
        </h1>
      </div>

      {user ? <button onClick={() => logoutUser()}>Déconnexion</button> : <a href="/auth/login">Connexion</a>}

      <nav className="w-1/2 hidden min-[769px]:block">
        <ul className="flex flex-wrap justify-between gap-4 w-full mx-auto">
          {navItems.map((item) => (
            <li key={item.name} className="flex flex-1 text-center items-center justify-center">
              <Link
                href={item.href}
                title={item.name}
                className="w-2/3 text-center text-xl py-5 rounded-full hover:bg-white hover:text-layout"
              >
                {item.name}
              </Link>
            </li>
          ))}
        </ul>
      </nav>
    </header>
  );
}
```

L'interface a été stylisée intégralement avec Tailwind CSS, une bibliothèque utilitaire qui m'a permis de construire rapidement des composants épurés, responsives et maintenables sans avoir recours à des fichiers CSS séparés. Par exemple, le formulaire de connexion « FormLogin.tsx » repose sur les hooks React « useState » pour capturer les entrées utilisateur et appliquer dynamiquement les classes Tailwind pour la mise en page et les interactions visuelle (effets de survol, message d'erreur, etc).

Voici un extrait de ma page FormLogin.tsx où le hook « useState » est utiliser en plus de Tailwind :

```
"use client";

import { useState } from "react";
import { loginUser } from "@app/firebase/auth";
import Image from "next/image";
import Link from "next/link";

export default function FormLogin() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    await loginUser(email, password);
  };

  return(
    <>
      <main className="mt-35 mb-35 ">
        <div className="flex justify-around">
          <div className="flex ">
            <Image
              src="/logo/locafestiaclair.png"
              alt="logo de Loca Festia"
              width={500}
              height={500}
            />
          </div>
          <div className="flex items-center justify-center ">
            <div className="w-full max-w-md mr-10">
              <h1 className="text-2xl text-button text-center mb-8 ">Connectez-vous <br /> à votre compte</h1>

              <div className="flex justify-center space-x-6 mb-8">
                <button className="flex items-center justify-center w-32 h-12 border border-gray-300 rounded-full hover:bg-gray-100">
                  <Image
                    src="/icon/google.svg"
                    alt="logo de Loca Festia"
                    width={500}
                    height={500}
                    className="max-md:hidden h-5 md:h-6"
                  />
                </button>
                <button className="flex items-center justify-center w-32 h-12 border border-gray-300 rounded-full hover:bg-gray-100">
                  <svg viewBox="0 0 24 24" width="22" height="22">
                    <path
                      fill="#000"
                      d="M18.71 19.5c-.83 1.24-1.71 2.45-3.05 2.47-1.34.03-1.77-.79-3.29-.79-1.53 0-2.77-3.27.82-1.31.05-2.3-1.32-3.14-2.81-.91.65.03 2.47.26 3.64 1.98-.09.06-2.17 1.28-2.15 3.81.03 3.02 2.65 4.03 2.68 4.04-.03.07-.42 1.44-1.38 2.83M13 3.05-3.11z"
                    />
                  </svg>
                </button>
              </div>
            </div>
          </div>
        </div>
      </main>
    </>
  );
}
```

DOSSIER PROFESSIONNEL (DP)

Chaque élément de la page (champs, bouton, icônes, lien vers d'autres sections) a été conçu pour être accessible et responsive, assurant ainsi une bonne expérience utilisateur sur mobile comme sur ordinateur. Cette première phase statique a permis de poser des fondations solides avant d'intégrer les comportements dynamiques et les échanges avec la base de données.

Utilisation des breakpoints et classes adaptatives Tailwind :

Pour garantir une mise en page responsive, j'ai utilisé des classes adaptatives fournies par Tailwind CSS. Par exemple, la classe « max-[768px] :mx-auto » m'a permis de centrer certains éléments uniquement sur les petits écrans, tandis que « md :flex » a permis d'adapter la disposition à partir de la taille d'écran moyenne. Ces breakpoints m'ont permis de gérer efficacement le comportement des éléments selon la taille de viewport, sans avoir à écrire de media queries manuelles, ce qui a facilité la maintenance et l'évolution de l'interface.

2. Précisez les moyens utilisés :

Visual Studio Code
Next.js
terminal
Git

3. Avec qui avez-vous travaillé ?

Pour cette partie j'ai travaillé en binôme avec un collègue de formation

4. Contexte

Nom de l'entreprise, organisme ou association ► *Dev Project & Co*
Chantier, atelier, service ► Premier Sprint intégration d'élément statique
Période d'exercice ► Du : *09/04/2025* au : *30/06/2025*

5. Informations complémentaires (facultatif)

Activité-type 1 Développer la partie Front-end d'une application web ou web mobile

Exemple n°3 ► Développer la partie dynamique des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Réaliser un interface utilisateur dynamique

LocaFestia

Page Login :

Après avoir posé les bases statiques de l'interface, j'ai poursuivi le développement de LocaFestia en y ajoutant des comportements dynamiques permettant une interaction fluide entre l'utilisateur et l'application. Cela inclut la gestion des états via les hooks React, les interactions conditionnelles, les formulaires contrôlés, ainsi que les appels à des services comme Firebase.

Par exemple, pour la gestion de la connexion utilisateur, j'ai mis en place un formulaire de login dynamique utilisant le hook « useState » pour capturer les champs saisis (email et mot de passe), et « useEffect » pour surveiller l'authentification en temps réel avec « onAuthStateChanged » de Firebase. Lorsqu'un utilisateur est connecté, le header affiche automatiquement un bouton « Déconnexion » à la place du lien « connexion ». Ce comportement est géré dynamiquement selon l'état de l'utilisateur stocké dans un « useState ».

Enfin, j'ai utilisé le routing dynamique de Next.js pour rediriger les utilisateurs après certaines actions (par exemple : une redirection vers l'accueil après une connexion réussie). Cela contribue à une meilleure expérience utilisateur, avec des retours visuels immédiats.

Voici un extrait de code affichant dynamiquement l'état d'authentification :

```
const [user, setUser] = useState<User | null>(null);
useEffect(() => {
  const unsubscribe = auth.onAuthStateChanged((user) => {
    setUser(user);
  });

  return () => unsubscribe();
}, []);

{user ? <button onClick={() => logoutUser()}>Déconnexion</button> : <a href="/auth/login">Connexion</a>}
```

Page d'accueil (carrousel) :

Afin d'enrichir l'expérience utilisateur sur le site de LocaFestia, j'ai intégré un carrousel dynamique d'images en utilisant la bibliothèque Swiper.js combinée au Firebase Storage. Ce carrousel permet d'afficher automatiquement toutes les images présente dans le dossier carrousel du Storage, sans modifier manuellement le code lors de l'ajout ou la suppression d'images.

Lors du montage du composant React, une fonction « useEffect » est déclenchée pour :

- Lister tous les fichiers disponibles dans le répertoire du Storage grâce à « listAll() »,
- Récupérer les URLs de téléchargement de chaque image avec « getDownloadURL() »
- Les stocker dans un tableau via le hook d'état « useState ».

Ces URLs sont ensuite injectées dynamiquement dans le composant « Swiper », qui crée autant de SwiperSlide qu'il y a d'images. Le tout est stylisé avec Tailwind CSS pour conserver une cohérence visuelle avec le reste de l'interface.

Le carrousel est responsive, avec un affichage conditionnel du nombre d'images par vue selon la taille de l'écran, et accessible grâce aux flèches de navigation et à la pagination cliquable.

Voici un extrait de code illustrant le chargement dynamique d'images depuis Firebase :

```
// Get images from Firebase when component mounts
useEffect(() => {
  const fetchImages = async () => {
    try {
      setLoading(true);
      // Reference to Carrousel folder in Firebase Storage
      const storageRef = ref(storage, 'Carrousel');
      // List all files in storage
      const result = await listAll(storageRef);
      // If no images, show error
      if (result.items.length === 0) {
        setError("No images found in Storage");
        return;
      }
      // Get download URLs for all images
      const urls = await Promise.all(
        result.items.map(async (itemRef) => {
          const url = await getDownloadURL(itemRef);
          return url;
        })
      );
      setImageUrls(urls);
    } catch (error) {
      setError(error instanceof Error ? error.message : "An error occurred");
    } finally {
      setLoading(false);
    }
  };

  fetchImages();
}, []);
```

Voici comment les images récupérées sont ensuite intégrés dynamiquement dans un composant Swiper pour afficher un carrousel responsive :

```
<div className="p-10 bg-layout" id="slider">
  <Swiper
    modules={[Navigation, Pagination]}
    spaceBetween={20}
    breakpoints={{
      640: {slidesPerView: 1}, // 1 image under 640px
      1024: {slidesPerView: 3}, // 3 images from 1024px
    }}
    navigation // Show navigation arrows
    pagination={{ clickable: true }}
    loop={true}
  >
    {imageUrls.map((url, index) => (
      <SwiperSlide key={index}>
        <Image
          src={url}
          alt={`Image ${index + 1}`}
          width={500}
          height={300}
          className="object-cover"
          priority={index === 0}
        />
      </SwiperSlide>
    ))}
  </Swiper>
</div>
```

2. Précisez les moyens utilisés :

Next.js
Tailwind CSS
terminal

3. Avec qui avez-vous travaillé ?

J'ai travaillé en binôme pour cette partie

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► *Dev Project & Co*

Chantier, atelier, service ► Réalisation de la partie dynamique

Période d'exercice ► Du : *09/04/2025* au : *30/06/2025*

5. Informations complémentaires (facultatif)

Activité-type 2 Développer la partie Back-end d'une application web ou web mobile

Exemple n° 1 ▶ Développer des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Fonctionnalité de classement par équipe :

Dans l'application SportGuess, j'ai développé une page de classement permettant aux utilisateurs de consulter le rang des membres au sein de leur équipe, en fonction du score cumulé sur leurs pronostics. Cette page s'adapte dynamiquement à l'équipe actuellement sélectionnée par l'utilisateur.

Lors de l'ouverture de la page, l'application interroge Firestore pour déterminer l'équipe associée à l'utilisateur connecté « selectedTeamID ». Une fois cette information obtenue, un StreamBuilder est utilisé pour écouter en temps réel les données de cette équipe. Si aucune équipe n'est sectionnée, un classement général est affiché.

Le classement est généré à partir des documents des membres référencés dans le champ « members » de chaque document d'équipe. Chaque membre est ensuite récupéré individuellement pour afficher :

- Son nom d'utilisateur,
- Son avatar (photo de profil ou initiale),
- Son score total,
- Et le nombre de pronostics exacts sur le total de pronostics réalisés.

L'utilisateur voit également son propre rang dans une bannière fixe en base de page, avec une mise en avant visuelle via une bulle colorée. Les membres sont triés par score décroissant.

L'ensemble de l'interface est responsive au thème clair ou sombre, grâce au « ThemeProvider », et respecte la charte graphique de l'application via des couleurs personnalisées (comme greenColor ou blueButton).

Ce système de classement permet d'instaurer une dynamique de compétition saine entre les utilisateurs, tout en mettant en valeur les performances individuelles au sein d'une équipe.

```
// Récupérer l'équipe sélectionnée de l'utilisateur
final userDoc = await _firestore.collection('Users').doc(user.uid).get();
final selectedTeamId = userDoc.data()?['selectedTeamId'];
```

Ce bout de code permet d'obtenir l'ID de l'équipe que l'utilisateur a sélectionnée, stockée dans sa fiche utilisateur sur Firestore. C'est le point de départ pour charger un classement par équipe.

```
return FutureBuilder<List<DocumentSnapshot>>(  
    future: Future.wait(membersRefs.map((ref) => ref.get())),  
    builder: (context, snapshot) {  
        if (!snapshot.hasData) {  
            return Center(child: CircularProgressIndicator());  
        }  
        final users = snapshot.data!.where((doc) => doc.exists()).toList();  
        if (users.isEmpty) {  
            return Center(  
                child: Text(  
                    'Aucun utilisateur trouvé',  
                    style: TextStyle(  
                        color: isDarkMode ? Colors.white : Colors.black,  
                    ), // TextStyle  
                ), // Text  
            ); // Center  
        }  
        // Trie les utilisateurs par score décroissant (du plus grand au plus petit score)  
        users.sort((a, b) => ((b.data() as Map<String, dynamic>)[ 'score' ] ?? 0).compareTo((a.data() as Map<String,  
dynamic>)[ 'score' ] ?? 0));  
        return _buildRankingList(users, isDarkMode);  
    },  
); // FutureBuilder
```

Ce code permet de récupérer dynamiquement les membres d'une équipe à partir d'une liste de références « DocumentReference » en utilisant un « FutureBuilder ». Il effectue les requêtes en parallèle grâce à « Future.wait », filtre les documents valides, puis trie les utilisateurs par score décroissant avant de les afficher.

Un indicateur de chargement s'affiche pendant la récupération, et un message est prévu si aucun utilisateur n'est trouvé. Cette approche garantit un affichage à jour et pertinent du classement des membres de l'équipe, tout en assurant une bonne expérience utilisateur.

Page de discussion en équipe :

La page ChatPage permet aux utilisateurs de communiquer par messages texte au sein de leur équipe sélectionnée dans l'application SportGuess. Cette messagerie en temps réel est connectée à Firebase Firestore, ce qui garantit la mise à jour instantanée des messages et leur affichage dynamique dans l'interface. Lors de l'ouverture de la page, l'équipe actuellement sélectionnée par l'utilisateur est automatiquement détectée grâce à une écoute continue des changements dans le document utilisateur :

```
void _listenToTeamChanges() {  
    final user = FirebaseAuth.instance.currentUser;  
    if (user == null) return;  
  
    _userSubscription = FirebaseFirestore.instance  
        .collection('Users')  
        .doc(user.uid)  
        .snapshots()  
        .listen((snapshot) {  
            if (snapshot.exists) {  
                final selectedTeamId = snapshot.data()?['selectedTeamId'];  
                if (selectedTeamId != null && selectedTeamId != _teamId) {  
                    _loadSelectedTeam();  
                }  
            }  
        });  
}
```

DOSSIER PROFESSIONNEL (DP)

L'utilisateur peut envoyer un message seulement s'il est membre de l'équipe. Le message est ensuite enregistré en base et l'interface fait défiler automatiquement jusqu'au dernier message :

```
void _sendMessage() async {
  if (_messageController.text.trim().isEmpty) return;

  try {
    // Vérifier si l'utilisateur est membre de l'équipe
    final isMember = await _chatService.isUserInTeam(_teamId);
    if (!isMember) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Vous devez être membre de l\'équipe pour envoyer des messages'),
          backgroundColor: Colors.red,
        ), // SnackBar
      );
      return;
    }

    await _chatService.sendMessage(
      teamId: _teamId,
      message: _messageController.text.trim(),
    );
  }
}
```

2. Précisez les moyens utilisés :

Firebase Firestore
Librairie Flutter
Visual Studio Code
Terminal

3. Avec qui avez-vous travaillé ?

J'ai travaillé en binôme avec un collègue de formation

4. Contexte

Nom de l'entreprise, organisme ou association ► *Dev Project & co*

Chantier, atelier, service ► Réaliser des composants d'accès NoSQL

Période d'exercice ► Du : *10/04/2025* au : *30/06/2025*

5. Informations complémentaires (facultatif)

Activité-type 2 Développer la partie Back-end d'une application web ou web mobile

Exemple n° 2 ► Développer des composants métier côté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Composant métier

Dans le cadre du développement de composants métier côté serveur pour l'application SportGuess, j'ai conçu et implémenté la gestion complète des équipes, en particulier la logique d'intégration et de gestion des invitations via des liens ou QRCode. Cette fonctionnalité permet à un utilisateur de rejoindre une équipe en analysant et en validant automatiquement l'URL d'invitation contenant les paramètres nécessaires, puis en vérifiant la présence de l'équipe dans la base Firestore ainsi que l'appartenance préalable de l'utilisateur. J'ai développé un service métier réutilisable, « EquipeService », qui centralise la logique de traitement des invitations, la validation des données et l'ajout effectif de l'utilisateur au tableau des membres de l'équipe. Ce service gère également la sélection de l'équipe active en mettant à jour le document utilisateur, facilitant ainsi le filtrage dynamique des informations affichées dans l'application. Enfin, une gestion fine des erreurs et des retours utilisateur a été intégrée via des mécanismes de notification, assurant une expérience fluide et robuste. Cette approche modulaire et centralisée améliore la maintenabilité de la gestion des équipes côté serveur.

```
_linkSubscription = _appLinks.uriLinkStream.listen((Uri uri) {  
  _handleDeepLink(uri.toString());  
}, onError: (err) {  
  print('Erreur de deep linking: $err');  
});
```

Cet extrait montre la mise en place de l'écoute des liens entrant, permettant à l'application de détecter une invitation d'équipe via un lien.

```
final uri = Uri.parse(qrData);  
if (uri.host != 'join-team') {  
  ScaffoldMessenger.of(context).showSnackBar(  
    SnackBar(  
      content: Text('Action non reconnue'),  
      backgroundColor: Colors.red,  
    ), // SnackBar  
  );  
  return;  
}
```

Cette condition filtre les actions autorisées via le lien pour ne traiter que celles qui concernent l'ajout à une équipe

```
final teamDoc = await Firestore.instance
    .collection('equipes')
    .doc(teamId)
    .get();

if (!teamDoc.exists) {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
            content: Text('Équipe introuvable'),
            backgroundColor: ■ Colors.red,
        ), // SnackBar
    );
    return;
}
```

Ce bloc de code vérifie que l'équipe référencée dans le lein existe bien dans la base de données Firestore avant de continuer le processus.

2. Précisez les moyens utilisés :

Firestore
Librairie Flutter
Visual Studio Code
Terminal

3. Avec qui avez-vous travaillé ?

J'ai travaillé en binôme avec un collègue de formation

4. Contexte

Nom de l'entreprise, organisme ou association ► *Dev Project & Co*

Chantier, atelier, service ► Réaliser des composant métier côté serveur

Période d'exercice ► Du : *09/04/2025* au : *30/06/2025*

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Bac Professionnelle	Lycée Chateauneuf	2016

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] Nicolas Zinck ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis
l'auteur(e) des réalisations jointes.

Fait à Mehun-Sur-Yèvre le 16/06/2025

pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

ANNEXES

(Si le RC le prévoit)